LLM Conversation for Part 2:

I am creating a website for my CS1 class final project. It is a website that creates a practice schedule for singers and dancers. The first page is the home_page which displays a welcome message and has 3 buttons: Create a practice schedule, set my goals, and view my project. If you click on create a practice schedule, using the create_schedule route, you will be brought to the create_schedule_page which asks you 3 questions- what would you like to practice and you can select singing, dancing, or both, what is your experience level and you can select beginner, intermediate, or advanced, and how much time do you have and you can select 15, 30, 45, or 60 minutes. There are two buttons at the bottom of the page where one is the back home button that takes you back home using the back_home route and the generate my schedule button which using the generate_schedule route, takes you to the schedule_page. The schedule page displays the practice plan that is generated according to the options selected. It has the part of the practice (ex. warm up), a check box where you can mark it as completed, and how long that practice activity is. At the bottom of the page, there are two buttons which are save completed session which uses the save_completed_session route to bring you to the progress_page which shows your progress. The view my progress button uses the view_progress route to bring you to the progress_page. The progress_page lists your practice goals that you inputted and their staus, their practice statistics of how many sessions they completed, how and many minutes practiced. It also shows their practice history of how many off those sessions were for dance and how many were for singing. There are two buttons at the bottom, one which is create new schedule which uses the create_schedule route to bring you back to the create_schedule_page and the back home button which uses the back_home route to bring you back to the home page. Going back to the home page, the view my progress button uses the view_progress route to take you to the progress_page. The set my goals button uses the set_new_goal route to bring you to the goal_setting_page which asks you what your goal is and you can type it in, to choose your hobby of singing or dancing that the goal is for, how often you want to practice and you can select 1, 2, 3, 4, or 5 days/week, and how long you want to work toward this goal and you can select 1, 2, 3, or 4 weeks. There are 2 buttons at the bottom of the page. One is save my goal which uses the save_goal route to bring you to the progress_page which updates your goal. The other button is the back home button which uses the back_home route to bring you back to the home page. The State dataclass has these attributes: activity_type: str - Gets modified when the user selects Dance, Singing, or Both on the create_schedule_page

experience_level: str - Gets modified when the user selects Beginner, Intermediate, or Advanced on the create_schedule-page
available_minutes: int - Gets modified when the user selects 15, 30, 45, or 60 minutes on the create_schedule_page
practice_schedule: list - Gets modified when the program generates a schedule based on the user's preferences on the schedule_page
completed_activities: list - Gets modified when the user checks off activities as they complete them on the schedule_page
practice_sessions: list - Gets modified when the program add a record when the user saves a completed session on the schedule_page
practice_goal: str - Gets modified when the user enters their personal improvement goal on the goal_setting_page
goal_activity: str - Gets modified when the user selects whether their goal relates to dancing or singing on the goal_setting_page.
goal_frequency: int - Gets modified when the user selects how many days per week they want to practice on the goal_setting_page
goal_duration: int - Gets modified when the user selects how many weeks they want to work toward their goal on the goal_setting page.
My second dataclass is PracticeActivity with these attributes:
activity_name: str - Gets modified when the app creates an activity, such as "Vocal Warm-ups" or "Choreography"
activity_type: str - Gets modified when the activity is created as either Dancing or Singing
difficulty: str - Gets modified based on the user's selected experience level
duration: int - Gets modified when the app assigns a number of minutes to the activity
completed: bool - Gets modified when the user checks off the activity as completed and will change from False to True
The loop is used to iterate through practice_schedule in order for the website to display each activity's name, duration, and completion status when displaying all of the activities in the user's practice schedule.
If statements would be used to determine the schedule outputted to the user.
Can you help me to code this website? Let's start with the back_home route. I already have the dataclasses and the import statements coded.

Your setup (a `State` dataclass, routes, pages, and buttons that point at routes) looks like Drafter, so I'm assuming that. Correct me if you're using something else.

The `back_home` route is simple: it takes the current state and returns the home page, so nothing the user has entered gets lost.

python

```python
@route

def back_home(state: State) -> Page:

    return home_page(state)
```

It needs a `home_page` function to return, and since you describe the home page as its own page, here's a sketch of it:

python

```python
def home_page(state: State) -> Page:

    return Page(state, [

        Header("Welcome to the Practice Planner!"),

        "Build practice schedules for singing and dancing, and track your goals.",

        Button("Create a practice schedule", create_schedule),
```

```
        Button("Set my goals", set_new_goal),
        Button("View my progress", view_progress),
    ])
```

If Drafter expects your first route to be named `index`, add this so the app starts on your home page:

python

```python
@route
def index(state: State) -> Page:
    return home_page(state)
```

Here are a few notes:

- **Why a helper function:** `back_home` and `index` both need to show the home page, so putting the page in `home_page` avoids writing it twice.
- **State:** `back_home` doesn't change anything in `State`. Its only job is navigation.
- **Missing routes:** the buttons on `home_page` refer to `create_schedule`, `set_new_goal`, and `view_progress`, which don't exist yet. Drafter may complain until they're defined, so add them as stubs if you run the app before finishing.

Which route would you like next? I'd suggest `create_schedule`, since it builds the page where the user fills in `activity_type`, `experience_level`, and `available_minutes`.

Can we create the create_schedule route next? I am assuming we will also need a create_schedule_page helper function

Yes, you need a `create_schedule_page` helper. The `create_schedule` route just calls it, and later `back_home` and other routes can reuse it.

python

```python
@route
def create_schedule(state: State) -> Page:
    return create_schedule_page(state)



def create_schedule_page(state: State) -> Page:
    return Page(state, [
        Header("Create a Practice Schedule"),

        "What would you like to practice?",
        SelectBox("activity_type", ["Singing", "Dancing", "Both"], state.activity_type),

        "What is your experience level?",
```

```
        SelectBox("experience_level", ["Beginner",
"Intermediate", "Advanced"], state.experience_level),


        "How much time do you have (in minutes)?",
        SelectBox("available_minutes", ["15", "30", "45", "60"],
str(state.available_minutes)),


        Button("Back home", back_home),
        Button("Generate my schedule", generate_schedule),
    ])
```

Notes:

- How the values reach `State`: The first argument to each `SelectBox` is its name, and Drafter passes the selected value to the next route as a parameter with that same name. So `generate_schedule` is where `activity_type`, `experience_level`, and `available_minutes` get saved into `State`. This route only displays the form.
- Defaults: The third argument to `SelectBox` is the pre-selected option, so each default in your `State` dataclass must exactly match one of the options. For example, use `activity_type: str = "Singing"`, `experience_level: str = "Beginner"`, and `available_minutes: int = 15`. If a default isn't in the list, the box may not render correctly.
- Why `str(...)` on the minutes: The options are strings, but `available_minutes` is an `int` in `State`, so the default has to be converted to match.
- Button order: The buttons appear in the order you list them, so Back home comes first and Generate comes second, as in your design.
- Stub for `generate_schedule`: The page refers to this route, so add a placeholder if you want to run the app before we write it:

python

```python
@route
def generate_schedule(state: State, activity_type: str,
experience_level: str, available_minutes: int) -> Page:
    return create_schedule_page(state)  # placeholder
```

The type hints on those parameters matter, because that is how Drafter converts the minutes string back into an `int`.

Next I'd suggest `generate_schedule` itself. It's where your if statements decide which `PracticeActivity` objects go into `practice_schedule`, so it's the biggest piece of logic. Want to tackle that one?

Can you help me with generate_schedule? I know that it wouldbring you to schedule_page with the generated schedule but I think it would definitely need a helper function

Yes, a helper is the right call. Here is a plan that splits the work into three pieces:

- `generate_schedule` (the route) saves the user's choices into `State`, builds the schedule, and shows the page.
- `build_activities` (the helper) uses if statements to create the `PracticeActivity` objects for one activity type.
- `schedule_page` (the page helper) uses your loop to display each activity.

python

```python
@route
```

```
def generate_schedule(state: State, activity_type: str,
experience_level: str, available_minutes: int) -> Page:
    state.activity_type = activity_type
    state.experience_level = experience_level
    state.available_minutes = available_minutes
    state.completed_activities = []

    if activity_type == "Both":
        half = available_minutes // 2
        state.practice_schedule = (
            build_activities("Singing", experience_level, half)
            + build_activities("Dancing", experience_level,
available_minutes - half)
        )
    else:
        state.practice_schedule =
build_activities(activity_type, experience_level,
available_minutes)

    return schedule_page(state)
```

```
def build_activities(kind: str, level: str, minutes: int) ->
list[PracticeActivity]:

    warmup_time = minutes // 5

    cooldown_time = minutes // 5

    main_time = minutes - warmup_time - cooldown_time


    if kind == "Singing":

        warmup_name = "Vocal Warm-ups"

        cooldown_name = "Vocal Cool-down"

        if level == "Beginner":

            main_name = "Scales and Breath Control"

        elif level == "Intermediate":

            main_name = "Song Practice"

        else:


           main_name = "Performance Run-Through"

    else:
```

```
        warmup_name = "Dance Warm-up"
        cooldown_name = "Stretching"
        if level == "Beginner":
            main_name = "Basic Steps"
        elif level == "Intermediate":
            main_name = "Choreography"
        else:
            main_name = "Freestyle and Performance Practice"

    return [
        PracticeActivity(warmup_name, kind, level, warmup_time,
False),
        PracticeActivity(main_name, kind, level, main_time,
False),
        PracticeActivity(cooldown_name, kind, level,
cooldown_time, False),
    ]



def schedule_page(state: State) -> Page:
```

```
    content = [Header("Your Practice Schedule")]


    for i, activity in enumerate(state.practice_schedule):
        content.append(f"{activity.activity_name} ({activity.duration} minutes)")
        content.append(CheckBox(f"activity_{i}", activity.completed))


    content.append(Button("Save completed session", save_completed_session))
    content.append(Button("View my progress", view_progress))


    return Page(state, content)
```

Notes:

- Time split: Each session is 20% warm-up, 20% cool-down, and the rest main practice. For “Both”, the time is split in half between singing and dancing, using `available_minutes - half` for dancing so the total always matches what the user picked. You can change those ratios however you like.
- Dataclass order: `PracticeActivity(warmup_name, kind, level, warmup_time, False)` passes arguments in the order you listed them: `activity_name`, `activity_type`, `difficulty`, `duration`, `completed`. Make sure your dataclass declares them in that order.
- Why `build_activities` takes a `kind`: Calling it once for singing and once for dancing keeps the “Both” case from needing its own copy of all the if statements.

- Checkbox names: Each checkbox gets a unique name (`activity_0`, `activity_1`, ...) so Drafter can tell them apart. Reading those values back is the job of `save_completed_session`, so we'll handle that when we get there. It's the trickiest part of the project, because the number of checkboxes changes with the schedule.
- Stubs: `save_completed_session` and `view_progress` need to exist before the app will run, so add placeholders if you want to test now.

You can test this one now: go home, pick options, and confirm the schedule changes for each combination. Then I'd suggest `save_completed_session` next, since it connects the checkboxes to `completed_activities` and `practice_sessions`.

is there another way to do the loop? We haven't learned f"{activity.activity_name}

Yes. An f-string is just a shortcut for joining text and values together. You can get the same result by adding strings with `+`, as long as you convert any number to a string with `str()` first. I also swapped `enumerate` for a plain `range` loop, in case you haven't covered that either.

python

Python

```
def schedule_page(state: State) -> Page:
    content = [Header("Your Practice Schedule")]

    for i in range(len(state.practice_schedule)):
        activity = state.practice_schedule[i]
        content.append(activity.activity_name + " (" +
str(activity.duration) + " minutes)")
```

```
        content.append(CheckBox("activity_" + str(i),
activity.completed))


    content.append(Button("Save completed session",
save_completed_session))
    content.append(Button("View my progress", view_progress))


    return Page(state, content)
```

Notes:

- Why `str()`: `activity.duration` is an `int`, and Python won't add a number to a string with `+`. Wrapping it in `str()` turns, say, `9` into `"9"` so it can be joined.
- The loop: `range(len(state.practice_schedule))` gives you the numbers 0, 1, 2, ... one for each activity. Each time through, `i` is the position and `state.practice_schedule[i]` is the activity at that position.
- Why keep `i`: You still need the number so each checkbox gets a unique name (`"activity_0"`, `"activity_1"`, ...).
- If you've learned a simpler loop: A plain `for activity in state.practice_schedule:` works for displaying the names and durations, but then you'd need a separate counter variable for the checkbox names. The `range` version keeps everything in one place.

For the project requirement, this still counts as a loop that iterates through `practice_schedule` to display each activity's name, duration, and completion status.

Everything else from the last step stays the same. Want to move on to `save_completed_session` next?

Yes we can move on to save_completed_session

Here is `save_completed_session`.

python

```python
@route
def save_completed_session(state: State, activity_0: bool = False, activity_1: bool = False,
                           activity_2: bool = False, activity_3: bool = False,
                           activity_4: bool = False, activity_5: bool = False) -> Page:
    checked = [activity_0, activity_1, activity_2, activity_3, activity_4, activity_5]

    state.completed_activities = []
    minutes = 0

    for i in range(len(state.practice_schedule)):
        activity = state.practice_schedule[i]
        activity.completed = checked[i]
        if checked[i]:
```

```
            state.completed_activities.append(activity)
            minutes = minutes + activity.duration


    if minutes > 0:
        record = PracticeActivity("Practice Session",
state.activity_type,
                                  state.experience_level,
minutes, True)
        state.practice_sessions.append(record)


    return progress_page(state)
```

Notes:

- Why six parameters: Each checkbox on `schedule_page` is named `activity_0`, `activity_1`, and so on, and Drafter passes each one to the route under that name. A schedule has 3 activities (singing or dancing) or 6 (both), so six is the maximum. The `= False` defaults cover the checkboxes that don't exist on a 3-activity schedule.
- The list `checked`: Putting the six values in a list lets the loop use the same index `i` for both the activity and its checkbox.
- What the loop does: For each activity, it updates `completed` from its checkbox. If it was checked, it's added to `completed_activities` and its minutes are added to the running total.
- What `practice_sessions` stores: I reused your `PracticeActivity` dataclass as a one-line summary of the session. Its `activity_type` is Singing, Dancing, or Both, and its `duration` is the minutes actually completed. This keeps the progress page easy. The number of sessions is `len(state.practice_sessions)`, the total minutes

come from adding up the `duration` values with a loop, and you can count dance and singing sessions with if statements. Treat a "Both" session as one of each.
- The `if minutes > 0` check: If the user checks nothing, no empty session is recorded.
- Things to test: I'm not certain how Drafter handles a checkbox that was left unchecked, so test a schedule with some boxes checked and some not. If you get an error about missing arguments, tell me the message and we'll adjust.
- Stub needed: This route returns `progress_page(state)`, which doesn't exist yet. It's also what `view_progress` will call.

That makes `progress_page` the natural next step. It's where the loops and if statements for the statistics go, plus the goal display. Want to build it next?

Yes

Here is `progress_page` along with the `view_progress` route. I added a small helper, `count_sessions`, because both the history and the goal status need to count sessions.

python

```python
@route
def view_progress(state: State) -> Page:
    return progress_page(state)



def count_sessions(state: State, activity_type: str) -> int:
    count = 0
    for session in state.practice_sessions:
```

```
        if session.activity_type == activity_type or
session.activity_type == "Both":
            count = count + 1
    return count



def progress_page(state: State) -> Page:
    total_sessions = len(state.practice_sessions)

    total_minutes = 0
    for session in state.practice_sessions:
        total_minutes = total_minutes + session.duration

    singing_sessions = count_sessions(state, "Singing")
    dancing_sessions = count_sessions(state, "Dancing")

    # Goal status
    if state.practice_goal == "":
        goal_text = "You haven't set a goal yet."
```

```
        goal_status = ""
    else:
        goal_text = state.practice_goal + " (" +
state.goal_activity + ")"
        target = state.goal_frequency * state.goal_duration
        done = count_sessions(state, state.goal_activity)
        if done >= target:

           goal_status = "Status: Completed!"
        else:
            goal_status = "Status: In progress (" + str(done) +
" of " + str(target) + " sessions)"

    return Page(state, [
        Header("Your Progress"),

        Header("Practice Goal", 2),
        goal_text,
        goal_status,
```

```
        Header("Practice Statistics", 2),
        "Sessions completed: " + str(total_sessions),
        "Minutes practiced: " + str(total_minutes),

        Header("Practice History", 2),
        "Singing sessions: " + str(singing_sessions),
        "Dancing sessions: " + str(dancing_sessions),

        Button("Create new schedule", create_schedule),
        Button("Back home", back_home),
    ])
```

Notes:

- `count_sessions`: It loops through `practice_sessions` and uses an if statement to count the sessions that match. A “Both” session counts toward singing and toward dancing, as we decided last step.
- Totals: The minutes come from a loop that adds up each session’s `duration`. The sessions count is just `len(...)`.
- Goal status: The rule I picked is that a goal is complete once the number of matching sessions reaches `goal_frequency * goal_duration`. For example, 3 days a week for 2 weeks is 6 sessions. This is my own design choice, so change it if your project handles goals differently.

- Matching names: The goal check compares `state.goal_activity` to the session types, so the goal page must use the same words, “Singing” and “Dancing”. Your State notes mention “Dance”, so pick one spelling and use it everywhere.
- State defaults: This page assumes `practice_goal` defaults to `""`, `practice_sessions` to an empty list, and `goal_frequency` and `goal_duration` to numbers like `1`. Check your dataclass for those.
- Header sizes: `Header("...", 2)` makes a smaller heading. If your version of Drafter doesn’t accept that, use plain text or remove the second argument.
- Only one goal: Your State holds a single `practice_goal` string, so the page shows one goal. That fits your design, but if you want a list of goals later, you’d change that attribute to a list.

You can now run the whole flow from the home page through creating a schedule, checking boxes, saving, and viewing progress. Left to build are `set_new_goal`, `goal_setting_page`, and `save_goal`. Want to start on those next?

Yes

Here are the three pieces for the goal page.

python

Python

```python
@route
def set_new_goal(state: State) -> Page:
    return goal_setting_page(state)



def goal_setting_page(state: State) -> Page:
    return Page(state, [
```

```
        Header("Set My Goals"),


        "What is your goal?",

        TextBox("practice_goal", state.practice_goal),


        "Which hobby is this goal for?",

        SelectBox("goal_activity", ["Singing", "Dancing"],
state.goal_activity),


        "How many days per week do you want to practice?",

        SelectBox("goal_frequency", ["1", "2", "3", "4", "5"],
str(state.goal_frequency)),


        "How many weeks do you want to work toward this goal?",

        SelectBox("goal_duration", ["1", "2", "3", "4"],
str(state.goal_duration)),


        Button("Save my goal", save_goal),

        Button("Back home", back_home),

    ])
```

```
@route
def save_goal(state: State, practice_goal: str, goal_activity:
str,
              goal_frequency: int, goal_duration: int) -> Page:
    state.practice_goal = practice_goal
    state.goal_activity = goal_activity
    state.goal_frequency = goal_frequency

   state.goal_duration = goal_duration

    return progress_page(state)
```

Notes:

- Same pattern as `create_schedule`: The page only displays the form, and the route saves the values into `State`. Each input's name (`"practice_goal"`, `"goal_activity"`, and so on) has to exactly match the parameter name in `save_goal`, because that's how Drafter connects them.
- Type hints: `goal_frequency: int` and `goal_duration: int` convert the selected strings back into numbers, which the progress page needs for `goal_frequency * goal_duration`.
- Defaults: The default for each `SelectBox` has to be one of its options. In your `State` dataclass, use something like `goal_activity: str = "Singing"`,

`goal_frequency: int = 1`, and `goal_duration: int = 1`. The goal text can default to `""`, which is what `progress_page` checks to decide whether a goal has been set.

- Spelling: The options say `"Singing"` and `"Dancing"` so they match the session types that `count_sessions` compares against. If you used `"Dance"` anywhere, change it to `"Dancing"` so goal progress counts correctly.
- Empty goal: If the user saves without typing anything, `practice_goal` stays `""` and the progress page shows "You haven't set a goal yet." You could add an if statement in `save_goal` to handle that differently, but it isn't required.
- Updating a goal: Saving a new goal overwrites the old one, which matches your "updates your goal" description. Goal progress counts all sessions so far, so it doesn't restart when a new goal is saved. If you want it to start fresh, you'd need to record the session count when the goal is saved.

That completes every route in your design. All the buttons should now point at real functions, so run through the whole app and check each page. If anything errors out, paste the error message and the code around it and I'll help you track it down.

Problem With the Route's Arguments

Your code asked a value for '.activity_type', but that value does not have anything with that name.

None

```
request.argument_parsing_failed: Error while parsing arguments
for request to URL index: type object 'State' has no attribute
'activity_type'
```

That error means your `State` dataclass doesn't have default values yet. Drafter creates a starting `State` when the app loads, and it can only do that if every attribute has a default. Without defaults, the first page (`index`) fails before anything is shown.

Here is `State` with defaults that match the code we've written:

python

Python

```python
from dataclasses import dataclass, field


@dataclass
class State:
    activity_type: str = "Singing"
    experience_level: str = "Beginner"
    available_minutes: int = 15
    practice_schedule: list = field(default_factory=list)
    completed_activities: list = field(default_factory=list)
    practice_sessions: list = field(default_factory=list)
    practice_goal: str = ""
    goal_activity: str = "Singing"
    goal_frequency: int = 1
    goal_duration: int = 1
```

Notes:

- Why the lists use `field(default_factory=list)`: Python doesn't allow a plain `= []` as a dataclass default, because every `State` would share the same list. `field(default_factory=list)` gives each `State` its own empty list. This is why

the import line `from dataclasses import dataclass, field` needs `field` added.

- Matching defaults: The `SelectBox` defaults on your pages must be one of the listed options, so `"Singing"`, `"Beginner"`, `15`, `1`, and `1` all work. `practice_goal = ""` is what `progress_page` checks to see whether a goal has been set.
- Check your decorator: `@dataclass` has to be directly above `class State:`. If it's missing, the attributes won't exist either, and you'd get this same error.
- Check how you start the server: The last line of your file should look like `start_server(State())`. If it is `start_server(State)` without the parentheses, you'd also get an error like this one.
- `PracticeActivity` doesn't need defaults: We always create it with all five values filled in, so it can stay as it is.

Once that's in, run it again. If you hit a new error, paste the whole message and the code around it and I'll help you sort it out.

So do I add this to State?

Yes, but replace your current `State` dataclass with it instead of adding it alongside. You should only have one `class State` in your file.

Your existing version probably looks something like this, with the attributes but no values:

python

Python

```python
@dataclass
class State:
    activity_type: str
    experience_level: str
    ...
```

Swap that whole thing for the new one, so each attribute gets an `= value` after its type. Also, update your import at the top so it includes `field`:

python

```python
from dataclasses import dataclass, field
```

Keep your `PracticeActivity` dataclass and everything else as is. Then run the app again. If a new error shows up, paste it here and we'll fix it.

```
Traceback (most recent call last):
  File "<string>", line 6, in <module>
  File "<string>", line 10, in State
NameError: name 'field' is not defined
```

Checking the import update for the Drafter library.

That error means Python doesn't know what `field` is yet, so the import at the top of your file doesn't include it. The new `State` uses `field(...)` for the lists, so it has to be imported first.

Find your dataclass import line near the top. It probably looks like this:

python

```python
from dataclasses import dataclass
```

Change it to:

python

```python
from dataclasses import dataclass, field
```

If you instead have `import dataclasses`, add this line below it:

python

```python
from dataclasses import field
```

Notes:

- Order matters: The import has to be above the `State` class in the file, since Python reads from top to bottom.
- Your Drafter import stays: Keep whatever line you already have for Drafter, such as `from drafter import *`. This only adds `field` to the dataclasses import.

Run it again after that. If something else comes up, paste the new error and I'll take a look.

i've never used field before, is there any other way to do this?

Yes, and it's probably the way your class has been teaching it. You don't need defaults inside the dataclass at all. You can leave `State` as plain attributes with no `= value`, and give the starting values when you start the server at the bottom of your file.

Your `State` goes back to this, with no `field` and no defaults:

python

```python
@dataclass
class State:
    activity_type: str
    experience_level: str
```

```
    available_minutes: int
    practice_schedule: list
    completed_activities: list
    practice_sessions: list
    practice_goal: str
    goal_activity: str
    goal_frequency: int
    goal_duration: int
```

The last line of your file creates a `State` with its starting values, in the same order as the attributes above:

python

```python
start_server(State("Singing", "Beginner", 15, [], [], [], "", "Singing", 1, 1))
```

You can also remove `field` from the import, so it's just `from dataclasses import dataclass` again.

Notes:

- Why this fixes your original error: The message said `type object 'State' has no attribute 'activity_type'`. That usually means `start_server(State)` was given the class itself instead of an actual `State` object with values filled in. Writing `State(...)` with the starting values creates that object.

- Order matters: The values go in the same order as the attributes. So `"Singing"` is `activity_type`, `"Beginner"` is `experience_level`, `15` is `available_minutes`, the three `[]` are the three lists, `""` is `practice_goal`, `"Singing"` is `goal_activity`, and the two `1`s are `goal_frequency` and `goal_duration`.
- Same starting values: I used the same values as before, so they still match the options in your select boxes.
- Start fresh: If you ran it earlier, restart the app so it uses the new starting state.

Run it again, and if there's another error, paste it here and we'll sort it out.

When I clicked on save completed session, I got this error Your code asked a value for '.acitivity_type', but that value does not have anything with that name.

None

```
request.route_execution_failed: Error while processing request
for URL 'save_completed_session': 'PracticeActivity' object has
no attribute 'acitivity_type'
```

That's a spelling typo: `acitivity_type` has the "i" and "t" swapped. Your `PracticeActivity` dataclass spells it correctly as `activity_type`, so the misspelled version is in the code that reads it.

Since the error comes up when `save_completed_session` finishes and shows the progress page, the most likely spot is the `count_sessions` helper:

python

Python

```
def count_sessions(state: State, activity_type: str) -> int:
    count = 0
    for session in state.practice_sessions:
```

```
        if session.activity_type == activity_type or
session.activity_type == "Both":

            count = count + 1

    return count
```

Check that both `session.activity_type` lines are spelled exactly like that. To find every copy of the typo, use your editor's search (Ctrl+F or Cmd+F) for `acitivity` and fix each match.

Notes:

- Spelling has to match everywhere: Python treats `acitivity_type` and `activity_type` as completely different names, so a typo in one place looks like a missing attribute.
- Also check: the `SelectBox` and route parameter names, such as `activity_type` in `create_schedule_page` and `generate_schedule`. They don't cause this exact error, but a misspelling there would break the connection between the form and the route.

After fixing it, restart the app and run through the flow again. If you get another error, paste it here with the code around it.

Now i am getting this error Your route function `save_completed_session` finished without returning a Page. In Python, a function that does not reach a return statement gives back None.

```
None

payload.verification_failed: Payload verification failed for the
page 'save_completed_session': The server did not return a
Page() object from save_completed_session.
```

```
Instead, it returned None (which happens by default when you do
not return anything else).

Make sure you have a proper return statement for every branch!
```

This error means some path through `save_completed_session` reaches the end of the function without hitting a `return`. The usual cause is indentation, so check where your return line sits.

The return has to line up with the other top-level lines of the function, not sit inside the `if`:

python

Python

```
   if minutes > 0:

        record = PracticeActivity("Practice Session",
state.activity_type,

                                  state.experience_level,
minutes, True)

        state.practice_sessions.append(record)


    return progress_page(state)
```

If your `return progress_page(state)` is indented one level further, it only runs when `minutes > 0`. When the user saves without checking any boxes, or the checkbox values don't come through as `True`, `minutes` stays `0`, the return is skipped, and Python gives back `None`.

Here is what to check:

- Return indentation: `return progress_page(state)` should start in the same column as `if minutes > 0:` and the `for` loop, with no extra spaces.
- No return at all: Make sure that line exists at the end of the function.
- The helper function: If `save_completed_session` looks right, check `progress_page`. The error names `save_completed_session`, but if `progress_page` is missing its `return Page(...)`, it would hand back `None` and you'd get the same message. The final lines should be `return Page(state, [...])`.

If the error shows up even when boxes are checked, the checkbox values may not be arriving as `True`. To test, check some boxes and click save. If the page still fails, paste your whole `save_completed_session` function here, with the indentation exactly as you have it, and I'll look at it.

I checked all of that stuff and it looks right but isn't working

Since the indentation checks out, the next most likely cause is a duplicate definition. In Python, if two functions share a name, the one lower in the file silently replaces the one above it.

Earlier I suggested placeholder stubs so you could run the app before everything was written, and those are easy to leave behind. If you have something like this lower in your file, it replaces your real function and returns nothing:

python

```python
@route
def save_completed_session(state: State):
    pass
```

To check, search your file (Ctrl+F or Cmd+F) for each of these names and make sure each appears in only one `def` line:

- `def save_completed_session`

- `def progress_page`
- `def view_progress`

If any of them shows up twice, delete the stub and keep the full version.

There is a second way to get this exact error. If `progress_page` is the one with the problem, `save_completed_session` can look perfect and still fail, because `return progress_page(state)` passes along whatever `progress_page` returns, including `None`. So also check that `progress_page` ends with `return Page(state, [...])`, and that this return isn't inside the `else` block of the goal-status section. It should line up with the `total_sessions = ...` line at the top of the function.

If none of that turns anything up, paste the full text of both `save_completed_session` and `progress_page`, exactly as they appear in your file, and I'll go through them line by line.

now it says this. I fixed the indentation. Your code used the name 'singing_sessions', but that name has not been created yet (or was misspelled).

None

```
request.route_execution_failed: Error while processing request for URL 'save_completed_session': name 'singing_sessions' is not defined
```

Python is saying that the name `singing_sessions` is being used on a line that runs before, or without, the line that creates it. This is a common side effect of fixing indentation, because moving a line can accidentally put it inside a loop or an `if`.

In `progress_page`, check the lines that create the counts. They need to sit at the same indentation as `total_sessions = ...`, not inside the `for` loop above them:

python

Python

```
def progress_page(state: State) -> Page:
```

```
    total_sessions = len(state.practice_sessions)


    total_minutes = 0
    for session in state.practice_sessions:
        total_minutes = total_minutes + session.duration


    singing_sessions = count_sessions(state, "Singing")
    dancing_sessions = count_sessions(state, "Dancing")
```

Here is what to look for:

- Indented too far: If `singing_sessions = ...` is indented under the `for` loop, it only gets created when a session exists, and it's created in the wrong place. It should line up with the `for` itself.
- Spelling mismatch: The name used in the page text has to match the one you created, letter for letter. For example, `singing_session` (no "s") in `"Singing sessions: " + str(singing_sessions)` would give this same kind of error. Search your file for `singing_session` and check every match.
- Defined after it's used: The two lines that create `singing_sessions` and `dancing_sessions` must come before the `return Page(...)` at the bottom, not after it.

If all of that looks right, paste your whole `progress_page` function and I'll look at it with you.

This is my function:
def progress_page(state: State) -> Page:
  total_sessions = len(state.practice_sessions)

```
total_minutes = 0
 for session in state.practice_sessions:
    total_minutes = total_minutes + session.duration

siging_sessions = count_sessions(state, "Singing")
 dancing_sessions = count_sessions(state, "Dancing")

#Goal status
 if state.practice_goal == "":
    goal_text = "You haven't set a goal yet."
    goal_status = ""
 else:
    goal_text = state.practice_goal + " (" + state.goal_activity + ")"
    target = state.goal_frequency * state.goal_duration
    done = count_sessions(state, state.goal_activity)
    if done >= target:
       goal_status = "Status: Completed!"
    else:
       goal_status = "Status: In progress (" + str(done) + " of " + str(target) + " sessions)"

return Page(state, [
    Header("Your Progress"),

   Header("Practice Goal", 2),
    goal_text,
    goal_status,

   Header("Practice Statistics", 2),
    "Sessions completed: " + str(total_sessions),
    "Minutes practiced: " + str(total_minutes),

   Header("Practice History", 2),
    "Singing sessions: " + str(singing_sessions),
    "Dancing sessions: " + str(dancing_sessions),
```

Button("Create new schedule", create_schedule),
Button("Back home", back_home),
])

Your indentation is fine. The problem is a typo on this line:

python

```python
siging_sessions = count_sessions(state, "Singing")
```

It's spelled `siging_sessions`, with the "n" missing, but the page below uses `singing_sessions`. Python sees them as two different names, so `singing_sessions` was never created.

Change it to:

python

```python
singing_sessions = count_sessions(state, "Singing")
```

Everything else in `progress_page` looks right, including the return, the goal section, and the buttons.

Since this is the second spelling typo in a row (`acitivity_type` earlier), a good habit is to copy and paste a variable name instead of retyping it, and to search the file for any name that gives you a "not defined" error. Search for `siging` after you fix this, in case it appears anywhere else.

Restart the app and run through the flow again. If something else breaks, paste the error and I'll help you track it down.

when i click on set new goal i get this error message Your code asked a value for '.goal_activity', but that value does not have anything with that name.

```
None

request.route_execution_failed: Error while processing request for URL 'set_new_goal': 'State' object has no attribute 'goal_activity'
```

Checking the State class for a missing or misspelled attribute.

That error means the `State` object doesn't have an attribute called `goal_activity`. Since the app starts and other pages work, the rest of `State` is probably fine, so the problem is likely in how that one attribute is written in your dataclass.

Check your `State` class and make sure this line exists, spelled exactly like this:

python

```
Python

    goal_activity: str
```

Here is what to look for:

- A misspelling: Your earlier typos were `acitivity` and `siging`, so look for something like `goal_acitivity` or `goal_activty`. Search the file for `goal_a` and compare every match.
- A missing line: It's possible the attribute was never added, since your original list of attributes was long. Compare your class against this full version:

python

```
Python

@dataclass
class State:
    activity_type: str
```

```
    experience_level: str
    available_minutes: int
    practice_schedule: list
    completed_activities: list
    practice_sessions: list
    practice_goal: str
    goal_activity: str
    goal_frequency: int
    goal_duration: int
```

- The server line: If you add or fix an attribute, the `State(...)` at the bottom of your file must have one value per attribute, in the same order. With the class above, it should be:

python

Python

```
start_server(State("Singing", "Beginner", 15, [], [], [], "",
"Singing", 1, 1))
```

Check that the `"Singing"` in the eighth spot lines up with `goal_activity`. If your class has the attributes in a different order, the values need to be in that same order.

After fixing it, stop and restart the app so it creates a fresh `State`. The error can keep showing up if the old version is still running.

If it still fails, paste your `State` class and your `start_server(...)` line and I'll compare them with you.

when i clicked on save goal this is what it said Drafter could not match the information sent by the page to the parameters of your 'save_goal' function.

```
None

request.argument_parsing_failed: Error while parsing arguments for request to URL save_goal: The route function 'save_goal' expects a value for parameter 'goal_frequency', but none was provided. Did you mean the request value 'goal_fequency'? Check that the form field or component argument is named 'goal_frequency'.
```

Another spelling typo. The error even suggests it: the form field is named `goal_fequency` (missing the "r"), but `save_goal` expects `goal_frequency`. Drafter matches form fields to route parameters by exact name, so the misspelled one never arrives.

The misspelling is in `goal_setting_page`. Find this line:

python

```python
SelectBox("goal_fequency", ["1", "2", "3", "4", "5"],
str(state.goal_frequency)),
```

and change the first argument to:

python

```python
SelectBox("goal_frequency", ["1", "2", "3", "4", "5"],
str(state.goal_frequency)),
```

Search your file for `fequency` afterward to make sure there are no other copies. The same typo could also be in your `State` class or in `progress_page`, though the error you got only points to the form field.

The name in quotes in each input (`"practice_goal"`, `"goal_activity"`, `"goal_frequency"`, `"goal_duration"`) has to match the parameter in `save_goal` letter for letter. It's worth comparing those four against the function definition once more while you're there.

Restart the app and try saving a goal again. If something else breaks, paste the error and I'll help you track it down.

Now that the app is working, can you help me to make it look pretty and not just like default drafter?

Yes. Drafter has a few built-in ways to change the look. I'll go from the easiest to the more custom, and each one is separate, so you can stop whenever you like. These function names come from Drafter's styling tools, and they can differ a little between versions. If one gives an error, paste it here and we'll adjust.

1. Title and debug panel

Put these near the bottom of your file, just above `start_server(...)`:

python

```python
set_website_title("Practice Planner")

hide_debug_information()
```

`hide_debug_information()` removes the developer panel that appears below your pages by default, which makes the biggest difference on its own. Leave it out while you're still fixing bugs, since that panel is what shows the state, and add it back when you're done.

2. A built-in style

python

```python
set_website_style("skeleton")
```

This swaps the whole look of the site (fonts, buttons, spacing) with one line. Some other names to try are `"water"`, `"sakura"`, `"mvp"`, `"picnic"`, and `"tacit"`. Try a few and keep the one you like best.

3. Colors on individual parts

You can wrap content in a `Div` and color it:

python

```python
Div(
    Header("Your Practice Schedule"),
    "Check off each activity as you finish it.",
)
```

then change its colors:

python

```python
change_background_color(Div(Header("Your Practice Schedule")), "lavender")
change_color(Div(Header("Your Practice Schedule")), "purple")
```

The first argument is the component and the second is the color name. You can use names like `"lavender"`, `"pink"`, or `"navy"`, or hex codes like `"#f5e6ff"`.

4. Page-wide custom styling

If you want the whole site to have a background color, you can add a CSS rule. Put this above `start_server(...)`:

python

```python
add_website_css("body", "background-color: #fdf2f8;")
```

The first part is what to style, and the second is the style itself. You can also pick a font with `font-family: Georgia;`.

Notes:

- Where things go: The calls in steps 1, 2, and 4 are written once, outside any function. Step 3 goes inside your page functions, on the pieces of content you want to style.
- Order of changes: Do step 2 first, then look at the pages. A good built-in style may be most of what you want, and anything you add on top of it will look more consistent.
- A suggestion for your theme: Since this app is for singers and dancers, a soft color palette like pinks and purples with `Georgia` as the font would fit it well. That's only a suggestion, so pick whatever you like.
- Test as you go: Make one change, restart the app, and look at it before adding the next. That way if something breaks, you know which line did it.

Which would you like to try first? I can help with the colors for a specific page, such as the home page.

Yes can we try to change the colors on the home page first?

Here's the home page with colors added. Everything you already have stays the same, and each styling function takes a component and a color, then hands the component back with the style applied.

python

Python

```python
def style_button(button):
    button = change_background_color(button, "#a855f7")
    button = change_color(button, "white")
    return button



def home_page(state: State) -> Page:
    welcome_box = Div(
        Header("Welcome to the Practice Planner!"),
        "Build practice schedules for singing and dancing, and track your goals.",
    )
    welcome_box = change_background_color(welcome_box, "#fdf2f8")
    welcome_box = change_color(welcome_box, "#6b21a8")
    welcome_box = change_padding(welcome_box, "20px")
    welcome_box = change_text_align(welcome_box, "center")
```

```
    return Page(state, [
        welcome_box,
        style_button(Button("Create a practice schedule",
create_schedule)),
        style_button(Button("Set my goals", set_new_goal)),
        style_button(Button("View my progress", view_progress)),
    ])
```

Notes:

- The welcome box: `Div` groups the header and text into one block so a single background color covers both. Then the four lines below it style that block: pink background, dark purple text, some breathing room inside, and centered text.
- `style_button`: This small helper gives all three buttons the same purple background and white text. Writing it once means that to change the button color later, you only edit the one function, and you can reuse it on other pages.
- Colors: The `#` values are hex codes. You can swap in color names like `"pink"` or `"navy"` if you like those better. Try changing a few and restarting to see what you like.
- If something errors: These styling functions can vary a little between Drafter versions. If one of them (like `change_padding` or `change_text_align`) says it isn't defined, delete that line and paste the error here. The look will still work without it.
- Debug panel: If you haven't yet, `hide_debug_information()` above `start_server(...)` removes the panel under the page, which makes the colors much easier to judge.

Restart the app, check the home page, and tell me what you think. If you like the direction, we can reuse `style_button` and the same colors on the other pages so everything matches.

is there a way to make the entire background that color instead of just the welcome box?

Weighing whether the background change should cover every page.

Yes. A `Div` only colors its own block, so for the whole page you need a CSS rule on the page's `body`. Put this once near the bottom of your file, above `start_server(...)`:

python

```python
add_website_css("body", "background-color: #fdf2f8;")
```

That's the same pink as the welcome box, so the box will blend in and the page will look like one color. You have a few options:

- Keep the box visible: Change the welcome box's `change_background_color` to a different shade, like `"white"` or a slightly darker pink such as `"#fbcfe8"`, so it stands out against the page.
- Let it blend in: Delete the `change_background_color(welcome_box, ...)` line. The text will sit directly on the page background.
- Add more to the rule: You can put several styles in the same string, like `add_website_css("body", "background-color: #fdf2f8; font-family: Georgia;")`.

Notes:

- It applies to every page: The `body` rule styles the whole site, so the schedule, progress, and goal pages will get the same background. For a consistent look that's usually what you want. If you'd rather have a different color per page, tell me and we can look at another approach.
- Where it goes: This is a one-time call outside any function, like `hide_debug_information()`. If you put it inside `home_page`, it would be re-added each time the page loads.
- If the color doesn't change: If you picked a built-in style with `set_website_style(...)`, it may override or compete with your rule. Try moving

the `add_website_css` line below `set_website_style`, and if it's still not working, paste your lines above `start_server` and I'll take a look.

Restart the app and check the home page. If you like it, the next step could be styling the other pages' buttons with `style_button` so they match.

do i need to remove anything from what we did before to the homepage?

No, nothing has to be removed. Everything from the home page code still works with the page-wide background. The only line you might change is the welcome box's own background color, and that depends on how you want it to look.

With the body set to `#fdf2f8` and the welcome box also set to `#fdf2f8`, the box blends into the page. That's fine if you like it, and you don't need to change anything. If you want the box to stand out, change that one line to a different shade:

python

```python
welcome_box = change_background_color(welcome_box, "white")
```

If you'd rather have no visible box at all, delete that line.

Everything else stays:

- `style_button` and the three styled buttons
- the text color, padding, and centering on the welcome box
- the `add_website_css("body", ...)` line above `start_server(...)`

One thing to double-check is that `add_website_css` appears only once in your file, outside any function. If you pasted it in a couple of times while experimenting, remove the extras.

If you like how the home page looks, the next step is to use `style_button` on the buttons in your other pages so everything matches. Want to do that?

for the practice schedule page, how can i make each part of the checklist on a different line?

Placing each activity and checkbox on its own line.

Drafter puts components next to each other by default, so the text and checkbox run together. The easiest fix is to wrap each activity's text and checkbox in a `Div`, which makes each activity its own block on its own line. Update the loop in `schedule_page`:

python

Python

```
def schedule_page(state: State) -> Page:
    content = [Header("Your Practice Schedule")]

    for i in range(len(state.practice_schedule)):
        activity = state.practice_schedule[i]
        label = activity.activity_name + " (" +
str(activity.duration) + " minutes)"
        content.append(Div(label, CheckBox("activity_" + str(i),
activity.completed)))

    content.append(Button("Save completed session",
save_completed_session))
    content.append(Button("View my progress", view_progress))
```

```
    return Page(state, content)
```

Notes:

- What changed: The text and checkbox used to be two separate items in the list. Now `Div(label, CheckBox(...))` groups them into one item, so the page shows one activity per line.
- Checkbox names: The names are unchanged (`"activity_0"`, `"activity_1"`, ...), so `save_completed_session` still works without any changes.
- Alternative: If you'd rather not use `Div`, you can add `LineBreak()` after each checkbox:

python

Python

```
       content.append(label)

        content.append(CheckBox("activity_" + str(i),
activity.completed))

        content.append(LineBreak())
```

This also puts each activity on its own line. If you get an error that `LineBreak` isn't defined, use the `Div` version.

- Styling: Since each activity is now a `Div`, you can style them like the welcome box. For example, wrap the `Div` in `change_background_color(..., "white")` and `change_padding(..., "10px")` to make each activity look like its own card. Tell me if you want that.

Restart the app, generate a schedule, and check that each activity shows on its own line.

can you help me make all of the pages look like the first one with the colors?

Yes. Since `body` already has the background color, what's left is giving the other pages the same title box and buttons as the home page. I'll add two small helpers so the styling is written once and reused everywhere. Put them next to your `style_button` function:

python

Python

```python
def style_title(box):
    box = change_background_color(box, "#fdf2f8")
    box = change_color(box, "#6b21a8")
    box = change_padding(box, "20px")
    box = change_text_align(box, "center")
    return box



def style_card(box):
    box = change_background_color(box, "white")
    box = change_color(box, "#6b21a8")
    box = change_padding(box, "10px")
    return box
```

If you changed the welcome box's background on the home page, use that same color in `style_title` so they match. Then update each page:

python

Python

```
def create_schedule_page(state: State) -> Page:
    return Page(state, [
        style_title(Div(Header("Create a Practice Schedule"))),

        style_card(Div("What would you like to practice?",
                       SelectBox("activity_type", ["Singing",
"Dancing", "Both"], state.activity_type))),
        style_card(Div("What is your experience level?",
                       SelectBox("experience_level",
["Beginner", "Intermediate", "Advanced"],
state.experience_level))),
        style_card(Div("How much time do you have (in
minutes)?",
                       SelectBox("available_minutes", ["15",
"30", "45", "60"], str(state.available_minutes)))),

        style_button(Button("Back home", back_home)),
        style_button(Button("Generate my schedule",
generate_schedule)),
    ])
```

```
def schedule_page(state: State) -> Page:
    content = [style_title(Div(Header("Your Practice Schedule")))]

    for i in range(len(state.practice_schedule)):
        activity = state.practice_schedule[i]
        label = activity.activity_name + " (" + str(activity.duration) + " minutes)"
        content.append(style_card(Div(label, CheckBox("activity_" + str(i), activity.completed))))

    content.append(style_button(Button("Save completed session", save_completed_session)))
    content.append(style_button(Button("View my progress", view_progress)))

    return Page(state, content)
```

```
def goal_setting_page(state: State) -> Page:
    return Page(state, [

       style_title(Div(Header("Set My Goals"))),

        style_card(Div("What is your goal?",
                       TextBox("practice_goal",
state.practice_goal))),
        style_card(Div("Which hobby is this goal for?",
                       SelectBox("goal_activity", ["Singing",
"Dancing"], state.goal_activity))),
        style_card(Div("How many days per week do you want to
practice?",
                       SelectBox("goal_frequency", ["1", "2",
"3", "4", "5"], str(state.goal_frequency)))),
        style_card(Div("How many weeks do you want to work
toward this goal?",
                       SelectBox("goal_duration", ["1", "2",
"3", "4"], str(state.goal_duration)))),

        style_button(Button("Save my goal", save_goal)),
```

```
        style_button(Button("Back home", back_home)),
    ])
```

For `progress_page`, keep all the code above the `return` exactly as it is (the totals, `singing_sessions`, and the goal status). Only replace the `return Page(...)` at the bottom:

python

Python

```
   return Page(state, [
        style_title(Div(Header("Your Progress"))),

        style_card(Div(Header("Practice Goal", 2), goal_text,
LineBreak(), goal_status)),
        style_card(Div(Header("Practice Statistics", 2),
                       "Sessions completed: " +
str(total_sessions), LineBreak(),
                       "Minutes practiced: " +
str(total_minutes))),
        style_card(Div(Header("Practice History", 2),
                       "Singing sessions: " +
str(singing_sessions), LineBreak(),
```

```
                        "Dancing sessions: " +
str(dancing_sessions))),


        style_button(Button("Create new schedule",
create_schedule)),
        style_button(Button("Back home", back_home)),
    ])
```

Notes:

- What's new: Every page now has a pink title box at the top, white cards for the content, and purple buttons, which matches the home page. The names inside `SelectBox`, `TextBox`, and `CheckBox` are unchanged, so none of your routes need to change.
- Change colors in one place: Because the colors live in `style_title`, `style_card`, and `style_button`, editing one of those three functions changes every page at once.
- Home page option: You could also use `style_title` on the home page's welcome box so it uses the same function as the others. That's optional.
- If something errors: `LineBreak` is the only component here that might not exist in your version. If you get an error that it isn't defined, delete the `LineBreak()` pieces and use a separate `Div` for each line instead. If it's one of the styling functions like `change_padding`, paste the error here.

Update one page at a time and restart to check it. If you'd like to adjust the colors, I can suggest a few other palettes.

what if i don't want the boxes to be white and i want to keep them the background color?

Weighing a style card's background color fix.

Just change the background line in `style_card`. You have two options.

Option 1: remove the line. The boxes then have no color of their own, so the page background shows through:

python

```python
def style_card(box):
    box = change_color(box, "#6b21a8")
    box = change_padding(box, "10px")
    return box
```

Option 2: set it to the same pink. This does the same thing visually, but it's useful if you ever change the page background and want the boxes to follow it:

python

```python
def style_card(box):
    box = change_background_color(box, "#fdf2f8")
    box = change_color(box, "#6b21a8")
    box = change_padding(box, "10px")
    return box
```

Notes:

- Keep the padding: The `change_padding` line still gives each section some breathing room, so the page won't look crowded even without visible boxes.

- Every page updates at once: Since all the pages use `style_card`, you only edit this one function.
- Make the sections stand out without white: If everything is the same color and the sections run together, you can use a slightly darker pink for the cards (like `"#fce7f3"`) instead of matching the background exactly. It's still subtle, but you can see where each section starts and ends.
- Title box: `style_title` already uses the same pink as the background, so it blends in too. If you want the title to be the only box that stands out, give it a different shade there.

Restart the app and check a couple of pages. If you'd like the colors changed to a different palette, I can suggest a few.

Can you help me write some assert_equal tests for the functions?

Yes. The best functions to test are the ones that take values in and return a result without drawing a page: `build_activities` and `count_sessions`. Those have the if statements and loops that matter most for your project. Put these below your function definitions and above `start_server(...)`.

If your class uses `assert_equal` from a library, make sure it's imported at the top. It's usually `from bakery import assert_equal`, but use whatever your course told you.

python

Python

```
# ---------- Tests for build_activities ----------


assert_equal(
    build_activities("Singing", "Beginner", 15),
    [
```

```
        PracticeActivity("Vocal Warm-ups", "Singing",
"Beginner", 3, False),
        PracticeActivity("Scales and Breath Control", "Singing",
"Beginner", 9, False),
        PracticeActivity("Vocal Cool-down", "Singing",
"Beginner", 3, False),
    ])


assert_equal(
    build_activities("Dancing", "Intermediate", 30),
    [
        PracticeActivity("Dance Warm-up", "Dancing",
"Intermediate", 6, False),
        PracticeActivity("Choreography", "Dancing",
"Intermediate", 18, False),
        PracticeActivity("Stretching", "Dancing",
"Intermediate", 6, False),
    ])


assert_equal(
    build_activities("Singing", "Advanced", 60),
```

```
    [
        PracticeActivity("Vocal Warm-ups", "Singing",
"Advanced", 12, False),
        PracticeActivity("Performance Run-Through", "Singing",
"Advanced", 36, False),
        PracticeActivity("Vocal Cool-down", "Singing",
"Advanced", 12, False),
    ])


assert_equal(
    build_activities("Dancing", "Beginner", 45),
    [
        PracticeActivity("Dance Warm-up", "Dancing", "Beginner",
9, False),
        PracticeActivity("Basic Steps", "Dancing", "Beginner",
27, False),
        PracticeActivity("Stretching", "Dancing", "Beginner", 9,
False),


   ])
```

```
# ---------- Tests for count_sessions ----------


singing_session = PracticeActivity("Practice Session",
"Singing", "Beginner", 15, True)

dancing_session = PracticeActivity("Practice Session",
"Dancing", "Beginner", 30, True)

both_session = PracticeActivity("Practice Session", "Both",
"Beginner", 45, True)


no_sessions = State("Singing", "Beginner", 15, [], [], [], "",
"Singing", 1, 1)

mixed_sessions = State("Singing", "Beginner", 15, [], [],

                       [singing_session, dancing_session], "",
"Singing", 1, 1)

with_both = State("Singing", "Beginner", 15, [], [],

                  [singing_session, both_session], "",
"Singing", 1, 1)

mostly_dancing = State("Singing", "Beginner", 15, [], [],

                       [dancing_session, dancing_session,
both_session], "", "Singing", 1, 1)
```

```
assert_equal(count_sessions(no_sessions, "Singing"), 0)

assert_equal(count_sessions(mixed_sessions, "Singing"), 1)

assert_equal(count_sessions(mixed_sessions, "Dancing"), 1)

assert_equal(count_sessions(with_both, "Singing"), 2)

assert_equal(count_sessions(with_both, "Dancing"), 1)

assert_equal(count_sessions(mostly_dancing, "Dancing"), 3)

assert_equal(count_sessions(mostly_dancing, "Singing"), 1)
```

Notes:

- Argument order: In `assert_equal(actual, expected)`, the first value is what your function returns and the second is what you expect. Many libraries accept either order, but this is the usual convention.
- How I got the expected numbers: They come from the time split we built. Warm-up and cool-down are each `minutes // 5`, and the main part is whatever is left. For 15 minutes that's 3, 9, and 3. For 60 minutes it's 12, 36, and 12. Working one out by hand before writing the test is the point of testing, so check one yourself to see that it matches.
- Variety of cases: The `build_activities` tests cover both activity types and all three experience levels, so every branch of your if statements runs at least once. For `count_sessions`, the tests include an empty list, mixed sessions, and the "Both" case.
- Why the test `State` objects: `count_sessions` takes a `State`, so each test builds one with the sessions list in the sixth spot, matching your attribute order.
- When a test fails: The message shows both values side by side. Either the function has a bug or your expected value was figured wrong, so check both.

Testing the routes themselves is harder because they return `Page` objects. If your instructor wants that too, ask them whether they expect route tests. I can help with that part, but the rules for it depend on how your course sets up Drafter.

can you explain the parameters for save completed session?

`save_completed_session` has seven parameters: `state` and six checkbox values.

python

```python
@route
def save_completed_session(state: State, activity_0: bool = False, activity_1: bool = False,
                           activity_2: bool = False, activity_3: bool = False,
                           activity_4: bool = False, activity_5: bool = False) -> Page:
```

`state: State` is the current `State` object. Drafter passes it to every route automatically, so you never provide it yourself. It's how the route reads `practice_schedule` and updates `completed_activities` and `practice_sessions`.

`activity_0` through `activity_5` are the checkboxes from `schedule_page`. The parameter names matter because of this line in that page:

python

```python
CheckBox("activity_" + str(i), activity.completed)
```

When `i` is 0, the checkbox is named `"activity_0"`. When the user clicks the save button, Drafter looks at every named input on the page and passes each value to the route parameter with the same name. That is why the names must match exactly, the same way `goal_fequency` broke `save_goal`.

`: bool` is the type hint. It tells Drafter to convert the checkbox value to `True` or `False`, the same way `: int` turned the select box strings into numbers in `generate_schedule`.

`= False` is the default value, used when no value arrives for that name. This matters because the number of checkboxes depends on the schedule. A Singing or Dancing schedule has 3 activities, so only `activity_0` to `activity_2` exist, and the other three parameters fall back to `False`. A "Both" schedule has 6 activities, so all six are used. Without the defaults, Drafter would raise the same "expects a value for parameter" error you got with `goal_frequency`.

Why six: It's the largest schedule the app can make. If you later let schedules have more activities, you'd need more parameters.

Inside the function, the first line gathers the parameters into a list:

python

Python

```
checked = [activity_0, activity_1, activity_2, activity_3,
activity_4, activity_5]
```

Then the loop uses `checked[i]` to look up the checkbox that belongs to `state.practice_schedule[i]`. That pairing by position is what links each activity to its checkbox.

I should flag one thing I wasn't sure about earlier. Some versions of Drafter don't send anything for an unchecked box. In that case the `= False` default is what makes unchecked boxes count as `False`, which is another reason the defaults are there.

`set_site_information` fills in the details about your project that appear on the deployed site. In the versions of Drafter I know of, it takes these keyword arguments, but check your assignment instructions, because your course may require specific values.

I am deploying this on github. What would I put for set_site_information()?

python

```python
set_site_information(
    author="your_email@school.edu",
    description="A website that builds practice schedules for singers and dancers, and tracks goals and progress.",
    sources=[
        "Drafter documentation",
        "Any other websites, people, or tools you got help from"
    ],
    planning="planning.pdf",
    links=["https://github.com/your-username/your-repo-name"]
)
```

Here is what each part is:

- **`author`:** Who made the site. Many courses want your school email here. Use whatever your instructor asked for.
- **`description`:** One or two sentences about what the site does. The one above comes straight from your project.
- **`sources`:** A list of strings naming where you got help, such as documentation, tutorials, classmates, or an AI assistant. If you used Claude for help with the code, check your course's academic integrity policy and list it if it requires disclosure.
- **`planning`:** The name or path of your planning document, such as a PDF of your page and route design. Leave it as an empty string `""` if your course doesn't ask for one.
- **`links`:** A list of related URLs. Your GitHub repository is a natural one.

Notes:

- **Where it goes:** Call it once, outside any function, above `start_server(...)`, next to `set_website_title` and `hide_debug_information`.
- **If you get an error:** If Python says an argument isn't recognized, your Drafter version may use different names. Paste the error here and we'll adjust.
- **Course requirements first:** If your instructor gave a template or a list of required fields, follow that over my example.

for the planning, how would I add the document?

The `planning` value is just the file name of your document. You put the document in your project folder and then type its name in that argument.

1. **Save your planning document as a file.** A PDF is the usual choice. In Google Docs or Word, use File > Download (or Save As) and pick PDF.
2. **Put it in the same folder as your Python file.** If your code is in `main.py`, the PDF should sit right beside it, not inside another folder.
3. **Use its exact name in the call:**

python

```python
set_site_information(

    author="your_email@school.edu",

    description="A website that builds practice schedules for singers and dancers.",

    sources=["Drafter documentation"],

    planning="planning.pdf",

    links=["https://github.com/your-username/your-repo-name"]

)
```

4. **Upload it to GitHub with the rest of your project.** On the repository page, click Add file > Upload files, drag the PDF in, and commit. If you use Git on your computer, `git add planning.pdf`, then commit and push.

Notes:

- **The name has to match exactly:** Capital letters, spaces, and the `.pdf` ending all count. If your file is `Practice Planner Plan.pdf`, then write `planning="Practice Planner Plan.pdf"`. Short names without spaces cause fewer problems.
- **If you put it in a subfolder:** Include the folder in the path, like `planning="docs/planning.pdf"`.
- **If you don't have one:** Use `planning=""` and leave it blank unless your course requires it.

- **Check the format your course wants:** Some instructors ask for a specific file type or contents, such as a diagram of your pages and routes. Follow what your assignment says.
- **If the link is broken after deploying:** Open your deployed site, click the planning link, and see whether the file loads. If not, the most common cause is that the file didn't get uploaded or its name doesn't match. Tell me what you see and we can track it down.